



Technical Report – Thermostat DHT22/ DHT11

T3 - Specialist Technician in Automation,
Robotics, and Industrial Control

Bruna Ferreira Gomes Madeira



Temperature Sensor (DHT22 / DHT11)

Purpose in the System

The DHT22/DHT11 sensor is used to measure the ambient temperature and relative humidity of the environment.. The measured data are transmitted to the microcontroller, which uses this information to perform control actions, such as activating or deactivating actuators in response to the environmental conditions.

Basic Operating Principle

Both the DHT22 and DHT11 sensors operate based on the same fundamental principle. They incorporate an NTC thermistor for temperature measurement and a capacitive sensor for relative humidity measurement. The analog signals generated by these sensing elements are internally converted into digital data and transmitted to the microcontroller through a single-wire digital communication protocol with strict timing control.

Differences Between DHT11 and DHT22

Despite their similar operating principle, the two sensors differ in performance. The DHT11 offers lower accuracy, reduced resolution, and a narrower operating range for both temperature and humidity measurements. Consequently, it is mainly suitable for low-cost and non-critical applications. In contrast, the DHT22 provides higher accuracy, better resolution, and a wider measurement range, making it more appropriate for applications that require greater reliability and precision.

LCD Display with I²C Interface

Role in the System

The LCD display is used to provide visual feedback to the user. It allows the presentation of relevant system information such as the measured temperature, system operating status (ON/OFF), setpoints, and possible warning or error messages.



Advantages of Using an I²C Interface

The use of an I²C interface significantly reduces the number of connections required between the microcontroller and the display, as only two communication lines (SDA and SCL) are needed. This simplifies the hardware design, reduces wiring complexity, and improves system reliability. Additionally, the I²C protocol allows multiple devices to share the same communication bus, enhancing system scalability.

Hysteresis Control Logic

Definition of Hysteresis

Hysteresis refers to the use of two distinct threshold values for switching an actuator ON and OFF, creating a tolerance band around the desired setpoint.

Importance in Temperature Control Systems

In temperature control applications, hysteresis is essential to prevent frequent and rapid switching of the output when the measured temperature fluctuates around the setpoint. This improves system stability, reduces mechanical and electrical stress on actuators such as relays, and increases the overall reliability and efficiency of the control system.

State Memory Variable

Purpose of Storing the Output State

A state memory variable is used to store the current output condition (ON or OFF). This approach avoids recalculating the output state solely based on instantaneous sensor readings at every loop cycle.

Benefits in Control Logic

Storing the output state allows correct implementation of hysteresis, prevents unstable behavior caused by noise or small measurement variations, and ensures predictable system operation. It also reduces unnecessary switching and improves control robustness.



Output Stage

Need for an Interface Stage

A relay should not be connected directly to a microcontroller pin because microcontroller outputs can supply only limited current and operate at low voltage levels. Furthermore, relay coils generate high-voltage transients when switched, which can damage the microcontroller.

Required Interface Components

An interface stage typically includes a transistor (BJT or MOSFET) to provide current amplification, a flyback diode to protect against inductive voltage spikes, and a base or gate resistor to control the transistor operation. In some applications, an optocoupler may be added to provide electrical isolation between the control and power circuits.

Reliability and Safety

Risks Associated with Sensor Failure

Failure of the temperature sensor, or the transmission of incorrect measurement data, can cause the control system to operate outside its intended conditions. Such failures may result in overheating or excessive cooling, unintended continuous operation of actuators, reduced system efficiency, and potential damage to equipment. In safety-critical applications, these conditions may also pose risks to users or the surrounding environment.

Methods to Improve System Safety

System reliability and safety can be enhanced through the implementation of basic fault-detection mechanisms, such as monitoring for invalid, missing, or out-of-range sensor readings. Defining a safe default operating state (fail-safe mode) ensures that the system enters a controlled condition in the event of a fault. Additionally, enforcing maximum and minimum temperature limits, using watchdog timers to detect software malfunctions, and applying sensor redundancy in critical applications further reduce the risk of unsafe operation.



Source Code – Temperature Control System

```
// Sensor DHT11 + LCD - New version with Potentiometer and enter Button

#include <Arduino.h>
#include "DHT.h"
#include <LiquidCrystal.h>

// --- LCD paralelo---
#define LCD_RS 12
#define LCD_E 11
#define LCD_D4 10
#define LCD_D5 9
#define LCD_D6 8
#define LCD_D7 7
LiquidCrystal lcd(12, 11, 10, 9, 8, 7);

// --- DHT ---
#define DHTPIN 6
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

// --- Saída ---
#define RELAY_PIN 3

// --- Interface de menu ---
#define POT_PIN A0
#define BTN_PIN 2 // Botão em INPUT_PULLUP (ENTER)

// --- Setpoints ---
float T0 = 24.0;
float T1 = 26.0;

// --- Estado saída ---
bool aquecimento = false;

// --- Debounce botão ---
bool btnLastRaw = HIGH;
bool btnStable = HIGH;
unsigned long tDebounce = 0;
const unsigned long DEBOUNCE_MS = 35;

// -----
// Converte leitura do potenciômetro (0..1023) para 5..70 °C
// (passos de 1 °C)
// -----
int potParaCelsius() {
```



```
int raw = analogRead(POT_PIN);      // 0..1023
int val = map(raw, 0, 1023, 5, 70); // 5..70
if (val < 5) val = 5;
if (val > 70) val = 70;
return val;
}

// -----
// Detecta "clique" (transição HIGH->LOW) com debounce
// Retorna true apenas 1 vez por clique
// -----
bool botaoClique() {
    bool raw = digitalRead(BTN_PIN);

    if (raw != btnLastRaw) {
        btnLastRaw = raw;
        tDebounce = millis();
    }

    if (millis() - tDebounce > DEBOUNCE_MS) {
        if (raw != btnStable) {
            btnStable = raw;
            if (btnStable == LOW) return true; // clique (premido)
        }
    }
    return false;
}

// -----
// Menu no arranque: define T0 e depois T1
// -----
void menuSetpoints() {
    // --- Definir T0 ---
    while (true) {
        int sel = potParaCelsius();

        lcd.setCursor(0, 0);
        lcd.print("Definir T0 (C) ");
        lcd.setCursor(0, 1);
        lcd.print("T0=");
        lcd.print(sel);
        lcd.print(" ENTER=OK "); // limpa resto

        if (botaoClique()) {
            T0 = (float)sel;
            break;
        }
    }
    delay(60);
}
```



```
}

// --- Definir T1 (obrigatoriamente > T0) ---
while (true) {
    int sel = potParaCelsius();

    int minT1 = (int)T0 + 1;    // minimo T0+1
    if (sel < minT1) sel = minT1;
    if (sel > 70) sel = 70;

    lcd.setCursor(0, 0);
    lcd.print("Definir T1 (C) ");
    lcd.setCursor(0, 1);
    lcd.print("T1=");
    lcd.print(sel);
    lcd.print(" ENTER=OK ");

    if (botaoClique()) {
        T1 = (float)sel;
        break;
    }
    delay(60);
}

// --- Mostrar resumo ---
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("Setpoints OK");
lcd.setCursor(0, 1);
lcd.print("T0=");
lcd.print(T0, 0);
lcd.print(" T1=");
lcd.print(T1, 0);

Serial.print(F("Setpoints definidos: T0="));
Serial.print(T0, 0);
Serial.print(F(" C, T1="));
Serial.print(T1, 0);
Serial.println(F(" C"));

delay(1400);
lcd.clear();
}

void setup() {
    Serial.begin(9600);
    Serial.println(F("Termostato Digital - DHT11"));
    Serial.println(F("Autora: Bruna M."));
}
```



```
dht.begin();

pinMode(RELAY_PIN, OUTPUT);
digitalWrite(RELAY_PIN, LOW);

pinMode(BTN_PIN, INPUT_PULLUP);

lcd.begin(16, 2);
lcd.clear();

lcd.setCursor(0, 0);
lcd.print("Termostato DHT11");
lcd.setCursor(0, 1);
lcd.print("A configurar...");
delay(900);

// --- Menu logo após o arranque ---
menuSetpoints();
}

void loop() {
  float temperatura = dht.readTemperature(); // °C
  float humidade    = dht.readHumidity();    // %

  if (isnan(temperatura) || isnan(humidade)) {
    Serial.println(F("Erro de leitura no DHT11"));
    lcd.setCursor(0, 0);
    lcd.print("Erro no sensor ");
    lcd.setCursor(0, 1);
    lcd.print("Verifique cabos ");
    delay(2000);
    return;
  }

  // --- Controlo com histerese ---
  if (temperatura > T1 && aquecimento) {
    aquecimento = false;
    digitalWrite(RELAY_PIN, LOW);
  }
  else if (temperatura < T0 && !aquecimento) {
    aquecimento = true;
    digitalWrite(RELAY_PIN, HIGH);
  }

  // --- Serial ---
  Serial.print(F("Temp: "));
  Serial.print(temperatura, 1);
```



```
Serial.print(F(" C Hum: "));  
Serial.print(humidade, 1);  
Serial.print(F("% T0="));  
Serial.print(T0, 0);  
Serial.print(F(" T1="));  
Serial.print(T1, 0);  
Serial.print(F(" Saida: "));  
Serial.println(aquecimento ? F("ON") : F("OFF"));  
  
// --- LCD 16x2 ---  
lcd.setCursor(0, 0);  
lcd.print("T:");  
lcd.print(temperatura, 1);  
lcd.print((char)223);  
lcd.print("C ");  
lcd.print("H:");  
lcd.print(humidade, 0);  
lcd.print("% ");  
  
lcd.setCursor(0, 1);  
lcd.print("T0:");  
lcd.print(T0, 0);  
lcd.print(" T1:");  
lcd.print(T1, 0);  
lcd.print(" ");  
lcd.print(aquecimento ? "ON " : "OFF");  
  
delay(1000);  
}
```

Simulation Resources

<https://wokwi.com/projects/452786512682196993>

Project Development Experience

At the beginning of the project, the instructor proposed a system based on a DHT22 temperature sensor and an LCD display with an I²C interface. The initial system concept and functionality were first validated through simulation using the Wokwi platform.

However, during the practical implementation phase, a discrepancy was identified between the simulated components and the hardware available in the laboratory kits. Instead of an LCD module with an I²C interface, a standard parallel LCD was available,



and the temperature sensor provided was a DHT11 rather than a DHT22. This situation required modifications to both the hardware connections and the software logic.

I was challenged by the instructor to adapt the circuit wiring and modify the source code based on the guidance provided. By carefully applying the given instructions and troubleshooting the differences between the components, I successfully implemented the required changes and achieved correct system operation. The adapted solution was then shared with my colleagues, enabling them to also assemble and test the project successfully.

After overcoming this initial challenge, the project was extended with the integration of additional components, including a potentiometer, a relay, and a push button. The potentiometer and the button were used to allow user configuration of the desired temperature setpoint, while the relay was introduced as the output switching element.

Following the successful implementation of these components, functional tests were carried out. In the first test, a lamp was connected to the relay to safely verify the correct ON/OFF switching behavior of the system. This test was successful and confirmed proper operation of both the control logic and the output stage. Finally, a heater was connected as the controlled load, and the system correctly activated and deactivated the heater according to the programmed temperature thresholds and hysteresis control logic. Both tests were completed successfully.

Overall, the project involved multiple technical challenges, from component adaptation to system expansion and testing. Successfully completing these stages was highly rewarding and significantly contributed to the development of practical skills in embedded systems, hardware–software integration, and structured problem-solving in an engineering context.